



38. Redundant Disk Arrays(RAID)

Only One Disk?

Solution 1: JBOD

Solution 2: RAID

Why Inexpensive Disks?

RAID

General Strategy: MAPPING

1) Dynamic mapping

2) Static mapping

General Strategy: REDUNDANCY

Reasoning About RAID

RAID

Workload

Metric

RAID Decisions

RAID-0

RAID-0: Striping

RAID-0: Analysis

왜 둘 다 N배인가?

RAID-1

RAID-1: Mirroring

Layout

RAID-1: 4 disks

Latent Sector Errors (LSE) vs. Silent Data Corruption

Latent Sector Errors (LSE) - 잠복 섹터 오류

Silent Data Corruption - 침묵적 데이터 손상(조용한 오류)

RAID-1: Analysis

RAID-1: Throughput

Crashes

H/W Solution

Raid-4

Raid-4: Strategy

RAID-4: Analysis

RAID-4: Throughput

RAID-5 Random Write $\approx R/2$

Why Not Read the Whole Stripe?

Why XOR Allows Incremental Parity Updates

Raid-5

RAID-5: Analysis

RAID-5: Throughput

RAID-4: Throughput과 비교

RAID Level Comparisons

Summary

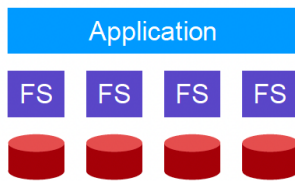
Only One Disk?

Sometimes we want many disks — why?

- capacity
- reliability
- performance

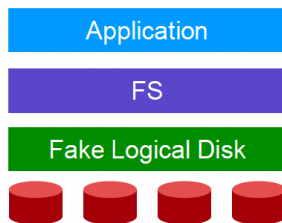
Challenge: most file systems work on only one disk

Solution 1: JBOD



- JBOD: Just a Bunch Of Disks
- Application is smart, stores different files on different file systems

Solution 2: RAID



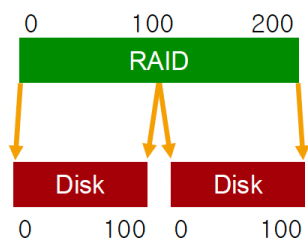
- RAID is: transparent, deployable
- Logical disk gives capacity, performance, reliability
- RAID: Redundant Array of Inexpensive Disks

Why Inexpensive Disks?

- Economies of scale! Commodity disks cost less
- Can buy many commodity H/W components for the same price as few high-end components
- Strategy: write S/W to build high-quality logical devices from many cheap devices
- Alternative to RAID: buy an expensive, high-end disk

RAID

General Strategy: MAPPING



Build fast, large disk from smaller ones

How should we map logical block addresses to physical block addresses?

1) Dynamic mapping

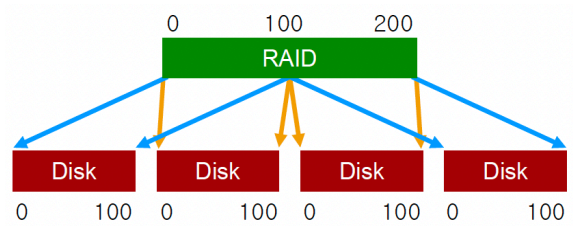
- use data structure (hash table, tree)
- page tables

2) Static mapping

- use simple math

- RAID

General Strategy: REDUNDANCY



- duplicated?
- Add even more disks for reliability
- 보통 3개까지 둬
- 쓰기 성능은 떨어질지도, 읽기 성능은 올라갈지도

Trade-offs to amount of redundancy

- Increase number of copies: improves reliability (and maybe performance)
- Decrease number of copies (deduplication): improves space efficiency

Reasoning About RAID

RAID

- system for mapping logical to physical blocks(0, 1, 4, 5, 6)

Workload

- types of reads/writes issued by applications (sequential vs. random)
- **Reads**
 - One operation
 - Steady-state I/O
 - Sequential
 - Random
- **Writes**
 - One operation
 - Steady-state I/O
 - Sequential
 - Random

Metric

- capacity, reliability(failed disk의 허용되는 개수), performance

Capacity (용량)

- how much space can apps use?

Reliability (안정성)

- how many disks can we safely lose? (assume fail stop!)

Performance (성능)

- how long does each workload take?
- Normalize each to characteristics of one disk

- N := number of disks
- C := capacity of 1 disk
- S := sequential throughput of 1 disk
- R := random throughput of 1 disk
- D := latency of one small I/O operation

Latency = 한 번의 작업을 끝내는 데 걸리는 시간

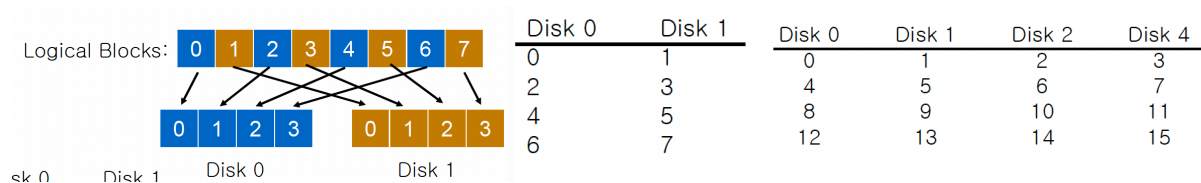
Throughput = 단위 시간 동안 얼마나 많은 작업을 처리할 수 있는가

RAID Decisions

- Which logical blocks map to which physical blocks?
- How do we use extra physical blocks (if any)?
- Different RAID levels make different trade-offs

RAID-0

RAID-0: Striping



- Optimize for capacity. No redundancy
- Given logical address A, find:
 - $\text{Disk} = A \% \text{disk_count}$
 - $\text{Offset} = A / \text{disk_count}$
- 동시에 2개를 쓸 수 있음, 읽기 속도도 좋음

Chunk size = 1

Disk 0	Disk 1	Disk 2	Disk 4
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Chunk size = 2

Disk 0	Disk 1	Disk 2	Disk 4
0 1	2 3	4 5	6 7
8 9	10 11	12 13	14 15

stripe: (8 9) (10 11) (12 13) (14 15)

RAID-0: Analysis

- Buying more disks improves throughput, but not latency!
- N := number of disks
- C := capacity of 1 disk
- S := sequential throughput of 1 disk
- R := random throughput of 1 disk
- D := latency of one small I/O operation
- What is **capacity**? → $N * C$

- 왜? 복제본이 없으니까 모든 디스크 용량 100% 사용 가능
- How many disks can **fail**? → **0**
- **Latency** → **D**
 - Latency = 작은 I/O 하나를 처리하는 시간
 - **RAID-0은 Latency를 개선하지 못하는 이유:** 작은 I/O는 결국 **하나의 디스크에만** 떨어짐
CPU가 명령을 여러 디스크에 "동시에" 보낼 필요도 없고 단일 요청은 1개 디스크만 움직임
- **Throughput (sequential일 때)** → **N * S**
 - 순차 읽기/쓰기에서는 여러 디스크가 동시에 일렬로 이어진 데이터를 읽기 때문
- **Throughput (random일 때)** → **N * R**

왜 둘 다 N배인가?

공통점: 병렬 처리

1. 순차 접근:

- 논리적으로는 "순차"이지만, RAID-0의 스트라이핑 때문에 물리적으로는 **여러 디스크에 분산 저장**
- 모든 디스크가 동시에 연속된 블록을 읽음

2. 랜덤 접근:

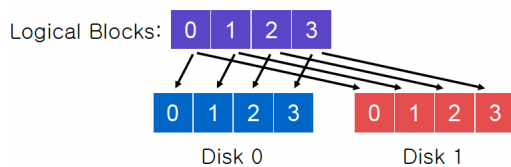
- 요청이 무작위로 들어와도 **각 디스크가 독립적으로 처리**
- 요청들이 여러 디스크에 분산됨



RAID-0은 throughput을 올리지만 latency는 못 올린다.

RAID-1

RAID-1: Mirroring



- striping 하면서 mirroring

Layout

	Disk 0	Disk 1
2 disks	0	0
	1	1
	2	2
	3	3

	Disk 0	Disk 1	Disk 2	Disk 4
4 disks	0	0	1	1
	2	2	3	3
	4	4	5	5
	6	6	7	7

RAID-1: 4 disks

- How many disks can fail?
- Assume disks are fail-stop.
 - each disk works or it doesn't

- system knows when disk fails
- Tougher Errors:
 - latent sector errors
 - silent data corruption

Latent Sector Errors (LSE) vs. Silent Data Corruption

Latent Sector Errors (LSE) - 잠복 섹터 오류

- A disk sector becomes corrupted, but the system does not notice immediately
디스크의 어떤 섹터(조각 데이터)가 망가져 있는데, 시스템이 즉시 알아차리지 못한다.
- The error is detected only when the sector is later read.
나중에 이 섹터를 읽으려고 할 때 비로소 "어? 못 읽는데?" 하고 발견
- Dangerous during RAID **rebuilt**, because a **single unreadable sector can cause the entire reconstruction to fail**
RAID 재빌드(rebuild) 중에 특히 위험함 → 재빌드는 모든 데이터가 정확히 읽혀야 하는데 한 섹터라도 못 읽으면 복구 전체가 실패할 수 있음.
- 왜 이런일이 생기나?
 - Causes include bit-rot in old HDD sectors
HDD 오래되면 **bit rot** (비트 썩음)
 - unreadable blocks discovered during RAID rebuilds
오랫동안 안 읽은 오래된 데이터 블록 부패
 - corruption in rarely accessed cold data
백업한 줄 알았는데 실은 이미 섹터가 망가져 있었던 경우



핵심: 데이터는 망가져 있었지만, 읽기 전까지 몰랐다 → "잠복(latent)" 오류

Silent Data Corruption - 침묵적 데이터 손상(조용한 오류)

- Data becomes corrupted, but the system fails to detect the problem, so it returns the corrupted data as if it were valid.
데이터가 손상되었는데, 시스템이 이를 전혀 감지하지 못한다
- This is the most dangerous type of storage error, because the corrupted data may be replicated, backed up, or used without detection.
- Causes include firmware bugs, DMA bit flips, driver bugs, or non-ECC memory errors.
펌웨어 버그, DMA 비트 플립, 드라이버 버그, Non-ECC 메모리

RAID-1: Analysis

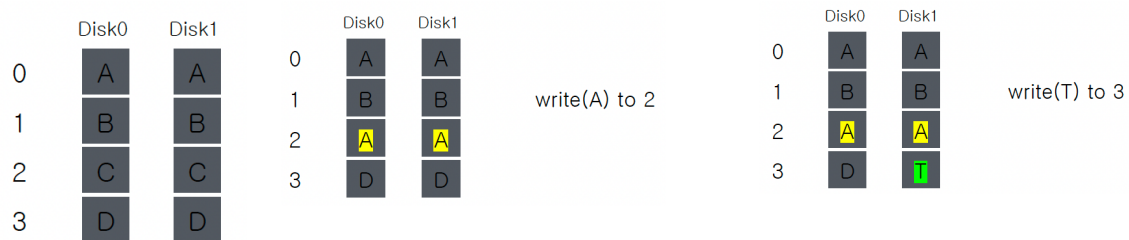
- N := number of disks
 C := capacity of 1 disk
 S := sequential throughput of 1 disk
 R := random throughput of 1 disk
 D := latency of one small I/O operation
- What is capacity? → $N / 2 * C$
- How many disks can fail? → **1 (or maybe $N/2$)**
- Latency → **D**
 - 읽기: 아무거나 하나만 읽으면 되니까 D
 - 쓰기: 두 개 써야 하지만 동시에 쓰니까 D

RAID-1: Throughput

- What is steady-state throughput for
- random reads? $\rightarrow N * R$
- random writes? $\rightarrow N / 2 * R$
 - 논리적으로 **1번의 쓰기 요청**이 발생하면, 실제로는 미러링된 2개의 디스크에 모두 써야 하므로 **2번의 물리적 쓰기**가 일어납니다.
 - 전체 물리적 디스크가 제공할 수 있는 총 IOPS는 $N * R$ 이지만, 논리적 쓰기 하나당 2배의 리소스를 소모하므로, 유효 성능은 절반인 $N / 2 * R$ 이 됩니다.
- sequential reads? $\rightarrow$ Book: $N / 2 * S$ (other models: $N * S$)
 - 순차성을 유지하기 위해 **한 쌍당 한 놈만** 주력으로 읽음
- sequential writes? $\rightarrow N / 2 * S$

Disk 0	Disk 1	Disk 2	Disk 4
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

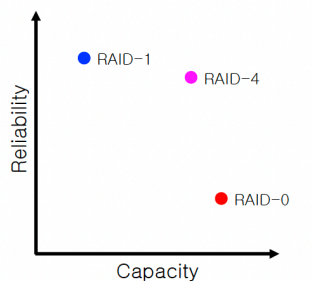
Crashes



- write(A) to 2
- write(T) to 3
- after reboot, how to tell which data is right?
- 나중에 이 블록을 읽으려 할 때, 어느 디스크가 정답인지 알 수 없게 되어 데이터가 손상될 위험에 처한다

H/W Solution

- Problem: Consistent-Update Problem
- Use non-volatile RAM in RAID controller
 - NVRAM (Non-Volatile RAM):** 전원이 꺼져도 데이터가 날아가지 않는 메모리. 보통 배터리가 달려있음
- Software RAID controllers (e.g., Linux md) don't have this option



- Reliability 측면: RAID-1 > RAID-4 > RAID-0
- Capacity 측면: RAID-0 > RAID-4 > RAID-1

Raid-4

Raid-4: Strategy

- Use **parity disk**
- In algebra, if an equation has N variables, and N-1 are known, you can often solve for the unknown.
- Treat sectors across disks in a stripe as an equation.
- Data on bad disk is like an unknown in the equation.
- XOR: 같으면 0 다르면 1

x1	x2	result
0	0	0
1	1	0
0	1	1
1	0	1

	Disk0	Disk1	Disk2	Disk3	Disk4	
Stripe:	5	3	0	1		(parity)
Stripe:	5	3	0	1	9	(parity)
Stripe:	5	X	0	1	9	(parity)
	2	1	1	X	5	(parity)
Stripe:	2	1	1	1	5	(parity)
Stripe:	3	0	1	2	X	(parity)
Stripe:	3	0	1	2	6	(parity)

- Which functions are used to compute parity?

RAID-4: Analysis

- N := number of disks
- C := capacity of 1 disk
- S := sequential throughput of 1 disk
- R := random throughput of 1 disk
- D := latency of one small I/O operation
- What is capacity? $\rightarrow (N - 1) * C$
- How many disks can fail? $\rightarrow 1$
- Latency $\rightarrow D, 2 * D$ (read and write parity disk)

RAID-4: Throughput

- What is steady-state throughput for
- sequential reads? $\rightarrow (N-1) * S$
- sequential writes? $\rightarrow (N-1) * S$
- random reads? $\rightarrow (N-1) * R$
- random writes? $\rightarrow R/2$ (read and write parity disk)

how to avoid
parity bottleneck?

Disk0	Disk1	Disk2	Disk3	Disk4
3	0	1	2	6

(parity)

RAID-5 Random Write $\approx R/2$

- Write-Modify-Write Penalty
 1. Read Old Data
 2. Read Old Parity
 3. Write New Data
 4. Write New Parity
- $\rightarrow 1 \text{ write} = 4 \text{ I/Os}$
- $\rightarrow \text{Effective random-write throughput} \approx R/2$

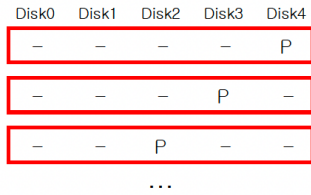
Why Not Read the Whole Stripe?

- Parity Can Be Updated Incrementally (XOR)
- Parity = $D0 \oplus D1 \oplus \dots \oplus Dn$
- New Parity = Old Parity $\oplus$ Old Data $\oplus$ New Data
- Implications:
 - No need to read all data blocks
 - **Only Old Data + Old Parity needed**
 - Efficient incremental parity update

Why XOR Allows Incremental Parity Updates

- Key XOR Property:
 - $A \oplus B \oplus B = A$ (duplicate terms cancel out)
- RAID-4 Parity:
 - Parity = $D0 \oplus D1 \oplus \dots \oplus Dk \oplus \dots \oplus Dn$
- When Dk changes to Dk' :
 - New Parity = Old Parity $\oplus$ Old Data (Dk) $\oplus$ New Data (Dk')
- Why?
 - Old Parity = (others) $\oplus Dk$
 - New Parity = (others) $\oplus Dk'$

Raid-5



- Rotate parity across different disks

RAID-5: Analysis

- N := number of disks
- C := capacity of 1 disk
- S := sequential throughput of 1 disk
- R := random throughput of 1 disk
- D := latency of one small I/O operation
- What is capacity? $\rightarrow (N - 1) * C$
- How many disks can fail? $\rightarrow 1$
- Latency $\rightarrow D, 2 * D$ (read and write parity disk)
 - All Same as RAID-4...

RAID-5: Throughput

- What is steady-state throughput for
- sequential reads? $\rightarrow (N-1) * S$
- sequential writes? $\rightarrow (N-1) * S$
- random reads? $\rightarrow (N) * R$
- random writes? $\rightarrow N * R/4$

RAID-4: Throughput과 비교

- What is steady-state throughput for
- sequential reads? $\rightarrow (N-1) * S$
- sequential writes? $\rightarrow (N-1) * S$
- random reads? $\rightarrow (N-1) * R$
- random writes? $\rightarrow R/2$ (read and write parity disk)

RAID Level Comparisons

- **RAID-5 is strictly better than RAID-4**
- RAID-0 is always fastest and has best capacity (but at cost of reliability)
- RAID-5 better than RAID-1 for sequential workloads
- RAID-1 better than RAID-5 for random workloads

	Reliability	Capacity
RAID-0	0	$C * N$
RAID-1	1	$C * N/2$
RAID-4	1	$(N-1) * C$
RAID-5	1	$(N-1) * C$

	Read Latency	Write Latency
RAID-0	D	D
RAID-1	D	D
RAID-4	D	2D
RAID-5	D	2D

	Seq Read	Seq Write	Rand Read	Rand Write
RAID-0	$N * S$	$N * S$	$N * R$	$N * R$
RAID-1	$N/2 * S$	$N/2 * S$	$N * R$	$N/2 * R$
RAID-4	$(N-1) * S$	$(N-1) * S$	$(N-1) * R$	$R/2$
RAID-5	$(N-1) * S$	$(N-1) * S$	$N * R$	$N/4 * R$

Summary

- Many engineering tradeoffs with RAID
 - capacity, reliability, performance for different workloads
- Block-based interface:
 - Very deployable and popular storage solution due to transparency

	capacity	reliability	read latency	write latency	seq read	seq write	random I/O
RAID0	$C * N$	0	D	D	$N * S$	$N * S$	$N * R$
RAID1	$C * N / 2$	1	D	D	$N / 2 * S$	$N / 2 * S$	$N * R$
RAID4	$C * (N - 1)$	1	D	$2 * D$	$(N - 1) * S$	$(N - 1) * S$	$(N - 1) * R$
RAID5	$C * (N - 1)$	1	D	$2 * D$	$(N - 1) * S$	$(N - 1) * S$	$N * R$

- N := number of disks
- C := capacity of 1 disk
- S := sequential throughput of 1 disk
- R := random throughput of 1 disk
- D := latency of one small I/O operation